Alma Mater Studiorum · Università di Bologna

Second Cycle Degree
Artificial Intelligence

Introduction to Complexity and Computability

Student:
Matteo Canghiari

Academic Year 2025/2026

Chapter 1

Conceptual and mathemetical preliminaries

First of all, we introduce the notion of **function**:

what a function really is?

Definition

Given two sets X and Y, such that:

$$X \cdot Y = \{(x, y) | x \in X, y \in Y\}$$

a **relation** is a subset of the cartesian product.

A function from X to Y is a relation described as:

$$f \subseteq X \cdot Y.$$

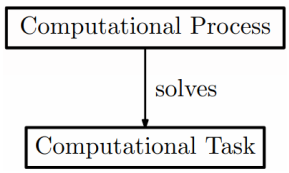
It's a subset of the cartesian product, such that $\forall x \in X$ there is one $y \in Y$ such that:

$$(x, y) \in f$$

and we state as follows: $f : x \to y$.

1.1 The standard paradigm

Consider the following image:



Processes and tasks are two different entities, that can be informally described as:

- **Computational process**: what we have to do, like a Python program.
- **Computational task**: what we really want to achieve.

There may be several processes for solving the same task, but sometimes the task is so hard to compute that no process can solve it. At the beginning, both of them are dependent from the **context**, but introducing a **model** (*arrow in the representation*) we are able to make them totally independent.

1.2 The multiplication problem

Example

e.g. Let's consider the following task.

Task: multiply two natural numbers a and b, both expressed by n digits.

A possible solution could be: reduce the multiplication to a sequence of additions. Therefore, knowing that we have to repeat the addition of number a for b times, our process is:

$$a \times b = a + a + \cdots + a$$

The amount of time required by this process can be summarized as follows:

$$b \times n$$

since we repeat the addition operation for b times. We call this process as **RepeatedAdditions**.

Another process that can solve the previous task might be: multiply a and b using the basic elementary school algorithm. Let $a_1, a_2, \ldots, a_n$ be the digits of a, we compute the final result performing $b \times (a_1 \times 10^0) + \cdots + b \times (a_n \times 10^{n-1})$, which takes a number of steps equal to $n \times n$. We call this process as **GridMethod**.

Between these two processes there's a huge difference, that can described as follows:

- RepeatedAdditions: the term b can be **exponential** in n, so b might be 10^n in the worst case.
- GridMethod: its complexity is quadratic, but anyway we are sure that it takes less time or effort than RepeatedAdditions.

Even though by both processes we can solve the task, RepeatedAdditions approach takes ages. Therefore, they solve the same task in different ways and

not always this is acceptable in terms of efficiency.

1.3 What is a process?

In theory of computation, a process is taken to be an **algorithm**, so we replace the term process by the word algorithm. Any algorithm must satisfy always these constraints:

1. It must be a **finite** description of computation steps.
2. Each step of the description must be as **elementary** as possible.
3. The way the next step is achieved must be **deterministic**.

1.4 Mathematical preliminaries

Definition

The cardinality of any set X is described by:

$$|X|$$

We will mainly work with discrete numbers: N the set of natural numbers and Z the set of integer numbers. Therefore, we avoid R the set of real numbers, since their cardinalities are infinite.

Definition

A condition $P(n)$ depending on $n \in N$ holds for n if there is a $N \in N$ such that $P(n)$ holds for every $n > N$.

We are defining a kind of *threshold* to determine if a property $P(n)$ for this n holds or not.

Definition

Given S a finite set, then a string (*a sequence of characters*) defined over an **alphabet** S is a finite, ordered, possible empty, tuple of elements coming from S.

Definition

The set of all strings over S of length exactly $n \in N$ is indicated as S^n, where S^0 is the set containing only the empty string ε.

Definition

The set of all strings over S is $\cup_{n=0}^{\infty} S^n$ and is indicated as S^*.

Definition

The concatenation of two strings x and y over S (*where x and y are defined over the alphabet S*) is indicated as:

$$xy \text{ or } x \cdot y$$

Definition

The **length** of a string is indicated as:

$$|x|$$

Example

e.g Let's consider the following alphabet.

$$\{0,1\}^2 = \{00, 01, 10, 11\}$$

where:

- $\{0,1\}^2 \rightarrow$ alphabet
- $\{00, 01, 10, 11\} \rightarrow$ strings defined over the current alphabet

A possible concatenation can be:

$$000101$$

whose cardinality is:

$$|000101| = 6$$

Useful notation

- For a given real number x, $\lceil x \rceil$ is the smallest element of Z such that $\lceil x \rceil \geq x$. Whenever a real number x is use in place of a natural number, we implicity read it as $[x]$.
- For a natural number n, $[n]$ is the set $\{1, \ldots, n\}$.
- $\log x$ is base 2 logarithm.

1.5 Tasks as functions

Summing up, we have stated:

1. A function is not a mapping operation, it is basically a relation between pairs composed by values coming from a domain and a codomain.
2. Any function is usually described by:

$$f : A \to B$$

   where:
   - $A \to$ domain
   - $B \to$ codomain
3. Sometimes A and B are countable not-string sets, therefore we must encode them following some strategies.
4. The encoding of any element x is indicated as $\lfloor x \rfloor$.

Typically, tasks (*what we want to achieve*) are about: given a function (*relation between two sets*) we turn an input to an output. In some cases, the input and output are not strings, so we need a way to represent them as a sequence of characters, following some encoding approaches.

Example

e.g. Let's consider the following alphabet.

$$S = \{a, b, c\}$$

any single character of this alphabet can be encoded as:

$a \to 00$
$b \to 01$
$c \to 10$

For instance, the original string *abbc* becomes:

$$00010110$$

Note: we get an injective function but not a surjective function, since 11 from the codomain corresponds to nothing in the domain.

Definition

Given any natural number n, we get an encoding equal to transform it in a binary number:

$$12 \rightarrow 1100$$

Definition

Given pairs of binary strings:

$$\{0,1\}^* \times \{0,1\}^*$$

where both $\{0,1\}^*$ can be a whatever sequence of binary numbers, we encode the resulting string $x\#y$ over the alphabet $\{0,1,\#\}$, where $\#$ acts as **separator**.

In computational theory, a task is interisting if it consists in computing a function from the alphabet $\{0,1\}^*$ to itself. An important function that starts from $\{0,1\}^*$ and goes to $\{0,1\}^*$ are those whose range are strings of length exactly one, named **boolean functions**, also called **characteristic functions**.

Definition

We identify the characteristic function such a function f with a subset L_f of $\{0,1\}^*$, defined as follows:

$$L_f = \{x \in \{0,1\}^* | f(x) = 1\}$$

Definition

Any subset of S^* (*the set of any string of any dimension* n) is called **language**, indicated as M.

These last definitions are extremely useful, because we can design a **decision problem** for a given language M (*subset of the alphabet* $\{0,1\}^*$) as task of computing a relation f such that the language $M = L_f$. If we can derive such equivalance we get a resolution of the problem.

Chapter 2

Computational model

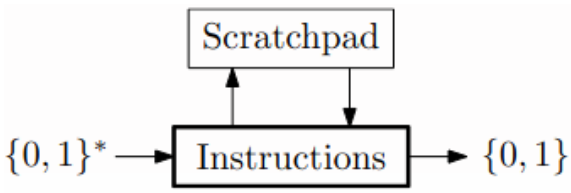
In order to prove our results we need a **computational model** in the form of an abstract machine, relying on two main assumptions:

1. It should be much **simple** as possible.
2. It must be able to run any other type of machine.

Our reference model will be the **Turing Machine**, that guarantees both the previous peculiarities.

2.1 The model informally

An easy representation about the Turing Machine could be as follows.



where:

- $\{0,1\}^* \rightarrow$ alphabet that provides us a whatever string of any dimension
- *Scratchpad* $\rightarrow$ physical part of the Turing Machine
- *Instructions* $\rightarrow$ set of symbols used by the Turing Machine
- $\{0,1\} \rightarrow$ answer of the Turing Machine as a single character

Therefore, given in input a whatever string from the alphabet $\{0,1\}^*$ returns 0 or 1, that respectively described when a **decision problem** is **decidable** or **undecid-**

able. Informally, if we want to compute a function $f : \{0,1\}^* \to \{0,1\}$, we need a finite, simple and generic set of instructions. Instructions of the Turing Machine are divided into:

1. Read a bit.
2. Read a symbol from the scratchpad.
3. Write a symbol into the scratchpad.
4. Declare the computation finished or terminated.

The **running time** of any Turing Machine on the input x is simply the number of basic instructions made. Generally, we say that the machine **runs in time** $T(n)$ if it performs at most $T(n)$ instructions on the input string of length n. Notice, every instruction made by the Turing Machine is done for each bit or *cell* that composed the original string x given in input.

Since the set of instructions is finite, the instructions can be encoded as strings, and therefore the machine can be described as a string.

Definition

Given a string α, we indicate with M_α the Turing Machine encoded by α.

Definition

With U we indicate the **Universal Turing Machine**, which simulates any other Turing Machine given its string representation (*as α in the previous definition*): given the pair (x, α) the machine U simulates the behaviour of M_α on x, where:

- $\alpha \to$ encoding of the Turing Machine M_α
- $x \to$ input string of whenever dimension

However, there are functions as $f : \{0,1\}^* \to \{0,1\}$ which are intrinsically uncomputable by Turing Machines. We need a more formal way in order to prove their **uncomputability**.

2.2 The model formally

Any Turing Machine is composed by a **scratchpad** and a set of **instructions**. More formally:

1st **Scratchpad**.
The scratchpad consists of k **tapes** ($k \geq 3$), each of them is a infinite one-directional

sequence of cells. Each cell stores a single symbol from a finite alphabet Γ, also called the alphabet of the machine. Every tape has also a **head**, capable of reading or writing one symbol at a time. These tapes are categorized into:

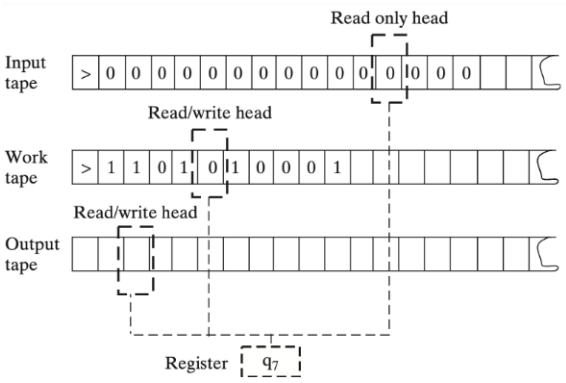
1. **Input tape**: typically read-only data structures, these store the initial input string x.
2. **Work tape**: intermediate tapes used for the computation of the input string x.
3. **Output tape**: tape that contains the result of the computation.

The tape alphabet Γ includes the following essential symbols:

- **Blank** symbol $\square$, indicates an empty cell.
- **Start** symbol $\triangleright$, marks the beginning of a tape.
- **0** and **1** symbols, since we are using strings.

2nd **Instructions**.
The machine operates through a finite set of states, Q. During each computational step, the machine performs the following sequence of actions:

1. **Read** the k symbols under the k tape heads.
2. For the $k-1$ read-write tapes, **replace** the current symbol with a new one, or leave it unchanged.
3. **Change** its state to a new one.
4. **Move** each of the k tape heads to the left, to the right or stay in place.



With these components defined, we have the necessary framework to construct a formal definition of any Turing Machine.

Definition

A **Turing Machine** working with k tapes is described by the triple (Γ, Q, δ), where:

- $\Gamma \rightarrow$ tapes' alphabet.
- $Q \rightarrow$ set of states, which includes an **initial state** q_{init} and a **final state** q_{halt}.
- $\delta \rightarrow$ **transition function**

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^{k-1} \times \{L, S, R\}^{k-1}$$

  The transition function maps the machine's current configuration to its next configuration, defined as:
  - $(Q \times \Gamma^k) \rightarrow$ the machine considers its current state $q \in Q$ and the k symbols currently read by the tape heads.
  - $(Q \times \Gamma^{k-1} \times \{L, S, R\}^{k-1})$:
    * $Q \rightarrow$ the machine shifts to a new state.
    * $\Gamma^{k-1} \rightarrow$ the machine updates the symbols of $k-1$ read-write tapes, since the input tape is read-only.
    * $\{L, S, R\}^k \rightarrow$ the machine specifies a direction (*Left, Stay, Right*) for each of the k tapes.

2.3 Machine configurations

Another important notion about any Turing Machine, described by the triple (Γ, Q, δ), is the **configuration**.

Definition

A **configuration** consists of:

- The current state q.
- The contents of k tapes.
- The positions of k tape heads.

There exists only one special configuration among the others, named the **initial configuration**.

Definition

The **initial configuration** I_x for the input $x \in \{0,1\}^*$ is the configuration in which:

- The current state is q_{init}.
- The first tape contains $\triangleright x$, followed by blank symbols $\square$, while the other tapes contain $\triangleright$, followed by blank symbols $\square$.
- The tape heads are positionated on the first symbol of the k tapes.

Additionally, there's also a **final configuration** that does not require any particular property as before.

Definition

A **final configuration** for the output $y \in \{0,1\}^*$ is any configuration of the Turing Machine whose state is q_{halt}, in which the content of the output tape is $\triangleright y$, followed by blank symbols $\square$.

2.4 Machine computations

...

2.5 ...

Chapter 3

Polynomial time computable problems

3.1 Complexity classes

A **complexity class** is a set of *tasks* which can be computed within a specific quantity of resources (roughly speaking, these tasks are sharing the same complexity, how complex they are to solve).

Note: a **computational task** is what we really want to achieve and it can be expressed as a function or a decision problem.

Typically, the tasks we are interested in are decision problems, or equivalently languages (for instance, a subset of the alphabet $\{0,1\}^*$). In particular, let $T : N \to N$ be a function used to detect how much time it's required for solving an instance of whatever problem, we state the following definition.

Definition

A language L, subset of S^* ($L \in \{0,1\}^*$), is in the class $\mathbf{DTIME}(T(n))$ if and only if there is a *Turing Machine* deciding L and running in time $n \to c \cdot T(n)$ for some constant c.

With the above definition we are saying that there exist some *Turing Machines* capable of solving the given task in linear time. This is not an universal definition, maybe other Turing Machines could require less time for the resolution. However, since a particular task has a time complexity **proportional** to $T(n)$ we know for sure that even in the worst case it requires no more than linear time.

The letter **D** in **DTIME**(.) refers to *determinism*: the machines on which the class is based work deterministically, swapping to an already known new configuration C_i.

3.2 The class P

The complexity class at the bottom of our hierarchy is **P**. The class **P** is defined as follows:

$$P = \bigcup_{c \geq 1} \mathbf{DTIME}(n^c)$$

We get the union of all the **DTIME**(n^c) classes whose function is any polynomial time representation n^c, therefore obtaining an unique set of all those tasks sharing the same time complexity.

In other words, the class **P** includes all those languages L (decision problems) that can be decided (solved) by a Turing Machine in a running time P, where P can be any polynomial time representation of the form n^c.

More formally, for any polynomial **P** there are c, d natural numbers that are strictly greater than 0 such that $P(n) \leq c \cdot n^d$ for suffiently large n. By this last statement, we can observe that c and d can be arbitraly large or huge, so, for instance, a *Turing Machine* deciding a given language L (decision problem) in time $10^{20} \cdot n^{10^{30}}$ still always a resolution in a polynomial time!

P is generally considered as **the class** of efficiently decidable languages, problems or tasks of this class seem to be simple, but anyway it's always possible to build a machine that solves the same problem or task with a different complexity.

3.3 The Strong Church-Turing Thesis

The Church-Turing Thesis

- Every physically realizable computer can be simulated by a Turing Machine with a overhead in time.
- The class of computable tasks would not be larger if formalized in a realistic way, but differently.

The Strong Church-Turing Thesis

- Every physically realizable computer can be simulated by a Turing Machine with **polynomial** overhead in time (n steps on the computer requires n^c on Turing Machines, where c only depends on the computer), and viceversa.
- The class **P** would be **the same** if defined based on other realistic models of computation.

Note: by the second thesis we specify that for any task belonging to the complexity class **P** its resolution by any Turing Machine always requires a *polynomial* overhead in time.

3.4 Why polynomials?

Why do we take the complexity class P as our main basis? There are three main reasons to do so, described as follows:

1. **P is robust**
   Polynomials seem to be the smallest class of bounds which makes P robust.
2. **Exponents are often small**
   The exponent c bounding the time of any Turing Machine capable to decide (solve) a language L (decision problem) such that $L \in \mathbf{P}$ can be huge. Anyway, in almost of the cases, the computational tasks solved have a complexity not greater than quadratic or cubic.
3. **Nice closure properties**
   It is relatively easy to prove that a given problem or task is **in** the class. Without constructing a Turing Machine explicitly, we can design an algorithm that guarantees the resolution of the problem in a polynomial running time.

3.5 The complexity class FP

Sometimes, one would like to classify **functions** rather than **languages** (remember, a computational task can be expressed by a function or a language). This can be done generalizing some concepts we have previously introduced.

In particular, let $T : N \to N$ be the function used to detect how much time it's required for solving an instance of whatever problem, we state the following definition.

Definition

A function f is in the class **FDTIME**(T(n)) if and only if **there is** a Turing Machine computing f and running in time $n \to c \cdot T(n)$ for some constant c.

The complexity class **FP** is defined as below, very similar to the complexity class **P**:

$$\mathbf{FP} = \bigcup_{c \geq 1} \mathbf{FDTIME}(n^c)$$

From the definition of the complexity class **FP** we can also describe other two peculiarities:

- For every language $L \in P$, the characteristic function f of L is trivially in **FP**.
- For certain classes of functions, there are canonical ways to turn a function f into a language L_f. However, it is not true that $f \in \mathbf{FP}$ implies $L_f \in \mathbf{P}$.

3.6 How to prove a task being in P or FP

In theory, we would design a Turing Machine, working within some polynomial boundaries, and prove that this machine decides (solves) the language (decision problem) or computes the function. However, this approach will take ages.

Instead of going through Turing Machines, we could use an informal but more expressive tool the so called **pseudocode**.

Example

e.g. Suppose we want to show the following problem to be computable in polynomial time:
given two strings $x, y \in \{0,1\}^*$ determine if x contains an instance of y (is y a substring of x?)

A pseudocode solving the problem above is the following.

```
i ← 1;
while i ≤ |x| − |y| + 1 do
    if x[i : i + |y| − 1] = y then
        return True
    else
        i ← i + 1
    end
end
return False
```

The most amount of work is done inside the *if condition*: by pseudocode we

cannot anymore state that every operation has the same unary cost as Turing Machines. In this case the amount of work done is at least equal to the size of y.

How could we be sure that the algorithm shown works in polynomial time? We must satisfy some constraints about that, divided into:

1. The input can be easily encoded as a binary string. Notice, we use pseudocode as a short-cut, but the formal proof should be done by the construction of a Turing Machine.
2. The total number of instructions executed is polynomially bounded.
3. All intermediate results are polynomially bounded in length.
4. Each instruction takes polynomial time to be simulated.

Example

e.g. Defining the polynomial complexity of the previous algorithm.

1. The total number of instructions executed is bounded by $O(|x|)$.
2. The intermediate results obtained over i cannot be greater than $O(|x|)$.
3. Comparing two strings of length $|y|$ can be done in polynomial time in $|y|$, thus polynomial in $|\lfloor(x, y)\rfloor|$

3.7 The class EXP

The next class of functions $T : N \to N$, beyond the polynomials and having nice closure properties is the class of exponential functions. The complexity classes **EXP** and **FEXP** are defined as follows:

$$\mathbf{EXP} = \bigcup_{c \geq 1} \mathbf{DTIME}(2^{n^c}) \ \mathbf{FEXP} = \bigcup_{c \geq 1} \mathbf{FDTIME}(2^{n^c})$$

The tasks in these classes can be solved mechanically, but possibly cannot be solved efficiently. Of course, the previous complexity classes **P** and **FP**, respectively for languages and functions, are subsets of the ones introduced.

$$\mathbf{P} \subseteq \mathbf{EXP} \ \mathbf{FP} \subseteq \mathbf{FEXP}$$

Theorem

The two inclusions above are strict.

The theorem seems to be quite weird, but what it's saying is quite easy to understand:

- Some problems are in **EXP** complexity class but not in **P**.
- Some functions are in **FEXP** complexity class but not in **FP**.